Zero Breach Toolkit

Security, privacy, malicious-behaviour, capability and engineering audit

Assessment date: 29 October 2026 | Supplied source: Zero-Breach-Toolkit-main.zip | Version declared in code: 0.1.11

Scope: static examination of the supplied archive, source code, release/build scripts, documentation, indicators and tests. No scanner, installer, test suite, third-party executable or network-enabled command was run. This is not a dynamic malware analysis, full formal verification or independently reproduced cross-platform assurance.

Executive summary — one page

OVERALL VERDICT: CONDITIONALLY SUITABLE FOR CAUTIOUS, OFFLINE, NON-PRIVILEGED EVALUATION; NOT CERTIFIED SAFE. The examined Python code exhibits an explicit read-only design and multiple meaningful safeguards. No clear, deliberately concealed exfiltration routine, persistence installer or unauthorised privilege-escalation mechanism was identified in the inspected source. That is a bounded negative finding, not proof that no backdoor exists.

Decision dimension	Assessment
Malicious intent / known backdoor	None established from static source inspection; residual risk remains
Unauthorised outbound transmission	No obvious scan-upload path; optional update, Android collection and browser test create external contact
Administrator/root access	Core flow is non-privileged; Windows PowerShell commands and optional elevated collection contexts merit review
Sensitive local information	High exposure if reports, evidence ZIPs, backup copies or temporary snapshots are mishandled
Reliability as an infection verdict	Limited: IOC and artefact checks cannot certify a device free of spyware
Third-party execution	Significant trust boundary: downloaded AndroidQF executable is launched by explicit command
Operational recommendation	Use verified source in a restricted, offline standard-user account; keep acquisition disabled initially

Most important identified risks are (1) the separately invoked download-and-execute Android collector; (2) potentially revealing local reports and unencrypted evidence bundles; (3) complete browser/iPhone database bytes in temporary snapshots, including data outside matching queries; (4) a Python-level socket guard that is not a system-wide network firewall; (5) optional browser navigation to external, non-project security-test domains; and (6) detection blind spots resulting in false reassurance.

Immediate safeguards: verify provenance; run as a standard user with OS-level network isolation; defer updates, Android collection and online browser tests; avoid cloud-synchronised output and protect or encrypt all evidence.

Risk ratings reflect operational exposure, not proof of malicious intent. Further validation requires controlled execution, network monitoring and independent review of third-party components.

1. Inventory, audit method and assurance limits

The uploaded ZIP SHA-256 is a9b7689137397250f6a9469f3b27369ac96d0f9d10830ed7084982bcced12f3d. The extracted package contains 214 regular files, including 162 Python files, substantial test fixtures and indicator lists. Static Python AST parsing found 0 syntax errors in the Python sources (syntax parsing alone does not validate imports or runtime behaviour).

Methods: ZIP inventory and file-type review; targeted grep across source, launchers and CI; manual inspection of update, collection, browser, guard, reporting, filesystem, decryption and platform code; dependency/version and provenance review; source-level identification of external processes, web access and deletion operations. No endpoint telemetry, packet capture, dynamic taint analysis, penetration testing or full line-by-line formal inspection was performed. External binaries, published wheels and source-origin signatures were not verified.

Trust boundaries

- User workstation → Python scanner → browser and operating-system artefacts → locally written report/evidence archive.
- User workstation → GitHub raw content for explicitly requested indicator updates; --latest changes freshness/supply-chain risk.
- User workstation → GitHub-hosted AndroidQF collector binary → USB-connected Android device; a separate executable and device privileges.
- User workstation → default web browser → archivegenocide.com or archivegenocide.org only if the online ad-blocking check is chosen.
- User workstation → operating-system file synchronisation, backups, antivirus, security agents or network-mounted storage outside the scanner's control.

2. Capability and platform matrix

Platform / source	Capabilities apparent in source	Limitations
Windows PC	Chrome-family, Firefox-family browser artefacts; host persistence, Defender context, PowerShell logs, event-related indicators, executable fingerprints and security status	Some records are privileged or missing; OS versions, permissions, locked files and collection errors create coverage gaps
macOS	Safari and other browser stores, launch agents/daemons, configuration and suspicious artefact traces	TCC/Full Disk Access restrictions; macOS alias paths; cannot prove live implant absence
Linux	Browser stores, known startup configuration and local traces	Distribution variance and permission restrictions; no universal endpoint sensor
Android phone	Analyses AndroidQF acquisition output for device policy, packages, suspicious app permissions and extracted traces	Not a live on-device Python scanner; USB debugging and separate collector required
iPhone / iPad	Reads local iOS backups or acquired filesystem/sysdiagnose; artefacts include messages and databases where accessible	Not direct iOS/iPadOS execution; backup completeness, encryption and sandbox constraints limit detection
Detection approach	IOC/domain/IP/path/hash matching, behavioural leads, optional YARA-X patterns, generated reports, STIX/MISP/denylist resources	Matches are investigative leads; unknown, novel and kernel-level malware may escape detection

3. Findings register — prioritised

F-01 | HIGH | Downloaded external collector is executed

Evidence: scan/zbtriage/phoneguide.py:26–75, 285–310

Observation and consequence: An explicit Android option downloads platform-specific AndroidQF v1.8.3 from GitHub release hosts, verifies a hard-coded SHA-256, marks it executable and runs it via subprocess with shell=False. Consent is requested. Hash pinning checks identity against the repository's expectation, not whether the collector is harmless or publisher-authenticated.

Recommendation: Treat Android collection as a separate product audit; independently verify release provenance/signature, audit AndroidQF source, perform sandboxed binary analysis and ask for separate informed consent.

F-02 | HIGH | Sensitive information is written locally

Evidence: PRIVACY.md:26–69; scan/zbtriage/report.py; scan/zbtriage/evidence.py

Observation and consequence: Full reports can include usernames, profile paths, browser context and message excerpts. Evidence ZIPs are unencrypted and original selected files may contain complete private databases. Synced folders can upload them without scanner exfiltration.

Recommendation: Default to secure local storage, private permissions, encryption, explicit retention period and redacted export; warn before cloud sync and avoid evidence export by default.

F-03 | HIGH | Temporary plaintext database exposure

Evidence: PRIVACY.md:49–64; scan/zbtriage/modules/browser.py; scan/zbtriage/ios_sources.py

Observation and consequence: Browser database snapshots and decrypted iPhone database copies may contain more than the queried fields; interrupted execution can leave plaintext snapshots in temporary directories.

Recommendation: Use a designated private scratch directory, secure permissions, minimal field extraction, OS-protected encrypted volume and robust cleanup; explicitly warn that deletion is not secure erasure.

F-04 | MEDIUM-HIGH | Offline enforcement is process-level, not OS-level

Evidence: scan/zbtriage/netguard.py:1–105; PRIVACY.md:5–9

Observation and consequence: Monkey-patching Python socket entry points blocks common Python networking but not subprocesses, native extensions, other processes, network filesystem traffic or all race conditions.

Recommendation: Add mandatory OS firewall deny policy or disposable VM network isolation for high-assurance scans; exercise egress denial tests with packet capture.

F-05 | MEDIUM | Browser test opens third-party web destinations

Evidence: scan/zbtriage/menu.py:149–195

Observation and consequence: A specifically confirmed menu option opens archivegenocide.com/scan/ or provides archivegenocide.org mirror. The external website and hosting parties may observe ordinary web traffic and browser data.

Recommendation: Separate this feature from the offline scanner, document domain ownership, avoid automatic recommendations and permit locally hosted tests.

F-06 | MEDIUM | PowerShell subprocess trust boundary on Windows

Evidence: scan/zbtriage/modules/host.py:200–230; modules/traces.py:95–115

Observation and consequence: Windows readers run PowerShell using subprocess and capture its output. This is legitimate collection functionality, but command construction, executable resolution and Windows policy need verification under hostile local state.

Recommendation: Use a verified system executable absolute path, constrained environment, strongly typed arguments, static command templates, script signatures and adversarial Windows tests.

F-07 | MEDIUM | Optional updates introduce supply-chain and connection risk

Evidence: scan/zbtriage/feeds.py:1–40, 403–431; PRIVACY.md:10–13

Observation and consequence: The --update flow downloads HTTPS content from an allow-list and normally uses pinned digests; --latest deliberately follows upstream branches, increasing change risk. HTTPS connections reveal IP and timing to infrastructure.

Recommendation: Require reproducible signed feed manifests; record each digest/source; restrict --latest; prefer off-host download and offline transfer for sensitive endpoints.

F-08 | MEDIUM | Detection confidence can be over-interpreted

Evidence: scan/zbtriage/__init__.py; README.md:200–300; scan/zbtriage/contract.py

Observation and consequence: A clean result means no supported known indicators were found in readable locations. Modern exploit chains, memory-only implants and tampered logs may produce no match.

Recommendation: Prominent “not a clean bill of health” statements, completeness scores by artefact/platform, independent MVT/forensic escalation and controlled positive/negative fixtures.

F-09 | MEDIUM | Locality checks leave residual filesystem races

Evidence: scan/zbtriage/localpaths.py:1–105; PRIVACY.md:5–9

Observation and consequence: Local path checks reject many symlinks, reparse points and remote shares, but path identity can change between checking and opening, and POSIX mounts cannot be fully controlled by lexical checks.

Recommendation: Use directory handles and openat-style no-follow access where available; recheck inode/device identity; sandbox filesystem access and test concurrent path swaps.

F-10 | MEDIUM | Evidence retention and deletion are not forensically sealed

Evidence: scan/zbtriage/evidence.py; PRIVACY.md:45–72

Observation and consequence: Source describes SHA-256 evidence integrity and temp-file replacement, but these do not create a signed custody record, prevent later local tampering or guarantee confidentiality.

Recommendation: Create signed provenance manifest, immutable append-only event log, UTC timestamps, optional encrypted evidence and reproducible validation tool.

F-11 | LOW-MEDIUM | Optional dependency exposure and compatibility

Evidence: requirements-patterns.txt; requirements-backups.txt; build/requirements-build.txt

Observation and consequence: Optional YARA-X and cryptography dependencies and build tooling expand the trusted computing base; hash pins help but runtime advisories, distribution authenticity and platform wheel coverage remain untested.

Recommendation: Generate SBOM, offline vulnerability scan, track advisories, verify package provenance and maintain supported Python/OS matrix.

F-12 | LOW-MEDIUM | Performance/resource exhaustion from hostile artefacts

Evidence: scan/zbtriage/readers.py; modules/browser.py; modules/patternscan.py; phoneguide.py:80–150

Observation and consequence: SQLite, ZIP, backup, large browser and pattern inputs are attacker-influenced. The code includes multiple bounds but untested worst cases can cause memory, CPU or disk exhaustion.

Recommendation: Enforce end-to-end time/byte/row ceilings and per-source budgets; profile large and malformed data; record skips without claiming completion.

4. Backdoor, spyware, malware and privilege review

No static evidence was established for a covert command-and-control beacon, unattended upload of scan results, hidden cron/task/service persistence installation, stealth credential theft, or self-executing privilege escalation in the core scan. This does not exclude malicious dependencies, trojanised release executables, attacker-modified archives or behaviours reachable only under specialised conditions. The ZIP contains source and scripts; no bundled platform-native executable was identified in the inspected inventory. AndroidQF is downloaded later on explicit request.

Administrator rights: the main CLI explicitly describes a standard-user workflow. It queries security-relevant configuration and launches PowerShell on Windows; some privileged records require administrative access and are reported as gaps or require separately invoked administrative checks. Reading whether an app is a device administrator is not equivalent to obtaining administrator privileges. Full Disk Access on macOS grants sensitive read access and should be granted only following an informed assessment.

Unauthorised transmission: the ordinary scan path has a Python socket guard and no obvious upload command. Network activity is explicitly possible through --update, --collect-android and the confirmed browser test. External applications, OS sync clients and mounted storage are outside that guarantee. “No observed source-level exfiltration” must not be presented as packet-capture-proven zero exfiltration.

5. Security engineering assessment

Protective measures actually visible

- Explicit no-network socket context manager, with blocked DNS and send/connect functions; clear statement of limitations (netguard.py).
- Fixed update-source allow-list, redirect checks, SHA-256 pins, size limits, cached manifest and fallback to old indicators (feeds.py).

- Android collector release host allow-list, SHA-256 pin, download size bound and shell=False launch (phoneguide.py).
- Locality/path guards against known symlink, reparse/remote-drive risks (localpaths.py).
- Explicit declaration of potentially sensitive output, plaintext temporary snapshots and non-encrypted evidence ZIPs (PRIVACY.md).
- Substantial source test coverage and release-focused CI workflow files exist; passing status was not independently established.

Threat-model gaps requiring independent verification

- Privilege abuse by a locally compromised Python interpreter or shell; library import shadowing and compromised site-packages.
- Supply-chain provenance of archive, tags, release workflows, downloaded binaries and dependency wheels.
- Network escape from native extensions, subprocesses, DNS/network filesystems and synchronisation clients.
- Unexpected disclosure through exception messages, console logs, clipboard, crash dumps or ordinary backups.
- Artifact parser confusion, traversal, symlink races, decompression bombs and malicious malformed input.
- Cross-platform compatibility of Windows junctions, Linux mounts, macOS TCC and encrypted mobile backups.

6. Privacy and information-flow audit

Data category	Collection / location	Primary handling issue
Browser history, site metadata, cookies	Local browser SQLite, caches, snapshots	Browsing profiling and potentially complete cookie databases in snapshots
Device and user identifiers	OS configuration, executable, system artefacts	Personally identifiable host and user metadata in report
iOS SMS/message fragments and backups	Selected backup, sysdiagnose or forensic collection	Personal messages; plaintext decryption copies and password memory
Android package/device-policy data	AndroidQF collector-generated local output	Third-party acquisition privileges and sensitive phone metadata
Selected evidence originals	Unencrypted local evidence ZIP	Exports may include full databases, not only detected indicators
Indicator downloads	HTTPS GitHub raw endpoints	IP address and timing disclosed to hosting infrastructure
Optional online browser test	External website in default browser	Web server and third parties may see standard browser requests

7. Performance and robustness review

Static review cannot establish measured throughput, RAM usage, device battery impact, disk overhead or crash rates. Browser copies, indexed IOC matching, recursive scanning, iOS database processing and optional pattern matching can be expensive. Source-level limits exist for collector downloads and extraction, but their efficacy under adversarial data has not been dynamically reproduced.

Area	Recommended metric / test	Acceptance objective

Runtime	Wall-clock by module, 95th/99th percentile on realistic fixture sizes	Predictable bounded completion, transparent timeouts
Memory	Peak RSS and allocations under large SQLite/ZIP/YARA inputs	Per-module caps and graceful degradation
Disk	Peak scratch/output size and interrupted-run residues	Preflight available-space check and cleanup manifest
Robustness	Corrupt databases, locked files, recursive links, malformed ZIP/TAR, unusual Unicode	No crashes, no boundary escape, explicit missing-coverage reporting
Accuracy	Known true-positive and false-positive evidence corpora	Per-platform precision/recall reports, no unjustified infection verdict
Portability	Windows 10/11, current macOS Intel/ARM, supported Linux distributions	Reproducible tested matrix and documented gaps
Exit semantics	Clean, match, incomplete, setup error, cancelled	Distinct exit statuses validated in CI

8. Improvement programme — prioritised

P0 — before routine use

- Audit and separately approve AndroidQF executable; fail closed if collector attestation is missing.
- Introduce optional enforceable operating-system egress quarantine for all offline workflows.
- Encrypt reports and evidence by default or avoid retaining complete artefacts.
- Use private scratch space and reliable interrupted-run cleanup with disclosure of secure-deletion limits.
- Document exact paths and destinations shown to the user before sensitive files are created.

P1 — next release

- Publish signed, reproducible source releases and SBOM; independent provenance verification.
- Add Linux/macOS/Windows hostile filesystem and subprocess escape tests.
- Subject parsers and IOC/YARA engines to coverage-guided fuzzing and memory/time ceilings.
- Strengthen evidence manifests with signature, custody metadata and reproducible verification.
- Add testable privacy modes: minimal/no-PII report, partial redaction, ephemeral analysis.

P2 — sustained quality

- Routine third-party vulnerability review and coordinated disclosure programme.
- Cross-platform CI on actual hosts, not merely synthetic mocks.
- Regular benchmark suite including worst-case and interrupted-state profiles.
- Versioned detection-quality studies against representative clean and infected corpora.
- Improve documentation on iPadOS restrictions, missing artefacts and limitations of a negative scan.

9. Safe operational procedure for first evaluation

1. Verify the ZIP SHA-256 recorded in this report against the exact file to be used and, separately, compare its GitHub tag/commit provenance with a trusted repository.

2. Inspect code in an ordinary text editor; do not invoke launchers with administrator privileges or bypass system security prompts on the strength of this report alone.

3. Use a dedicated standard-user account or disposable machine; initially prohibit all outbound traffic using an OS firewall or disconnected network.

4. Avoid --update, --latest, --collect-android and browser ad-block testing on the first run. Use pinned indicator data already in the archive.

5. Ensure the working directory, system temporary directory and findings output are local, not cloud-synchronised or network-mounted; use full-disk encryption.

6. Monitor processes, filesystem writes and network connections during a controlled test; inspect created reports, archives and scratch data before trusting the result.

7. If scanning iOS backups, work on an encrypted, access-controlled copy; restrict passwords to the private prompt and avoid evidence export unless necessary.

8. Treat every positive as an investigative lead and every negative as incomplete; for suspected targeted compromise use a qualified incident-response or mobile forensics team.

10. Evidence index and reproduction guidance

All relative paths below refer to the archive root. Line numbers refer to the supplied source snapshot and can change in later releases.

- PRIVACY.md — network, acquisition, sensitive output and temporary storage
- README.md — platform claims, operating instructions, Android and iPhone limitations
- scan/zbtriage/cli.py — scan orchestration, privileges and command-line paths
- scan/zbtriage/netguard.py — Python socket interception and limitations
- scan/zbtriage/feeds.py — network updates, allow-lists and digest pins
- scan/zbtriage/phoneguide.py — AndroidQF binary verification, extraction and launch
- scan/zbtriage/menu.py — external browser test URL and consent
- scan/zbtriage/localpaths.py — filesystem locality and symlink policy
- scan/zbtriage/modules/host.py and modules/traces.py — PowerShell execution
- scan/zbtriage/ios_sources.py and modules/browser.py — sensitive database processing
- scan/zbtriage/evidence.py and report.py — evidence/report generation
- requirements-backups.txt and requirements-patterns.txt — dependency hashes
- build/ and .github/workflows/ — assembly and release pipeline
- scan/tests/ — regression coverage (existence checked, execution not verified)

11. Final judgement and outstanding verification

Based solely on the supplied source, Zero Breach Toolkit does not demonstrate an obvious deliberate backdoor or hidden exfiltration mechanism in its ordinary scan flow. It nevertheless processes exceptionally sensitive local information and has explicit optional actions that contact external servers or run a downloaded third-party programme. It must be treated as untrusted software until provenance and runtime behaviour are independently verified. The risk of a missed intrusion is distinct from the risk that this toolkit itself compromises privacy; both must be managed.

Residual verification needed for a defensible exhaustive clearance: full line-by-line peer code review, signed-origin verification, third-party binary reverse engineering, dependency advisory assessment, sandboxed runtime observation with OS-level egress logging, fuzz testing, permission auditing and test execution across genuine target operating systems. No source-only report can certify the absence of every concealed backdoor or prove malware-free behaviour.